

Gerçek Zamanlı Meme Ultrason Lezyon Segmentasyonu

Üç mimarinin kalite ve hız karşılaştırması — SegFormer-B0, RTMDet-Ins-tiny, YOLOv11n-Seg

1. Amaç ve veri

Meme ultrasonunda lezyonun canlı görüntü akışı üzerinde segmentasyonu hedeflendi. Soru iki parçalı: *hangi mimari daha iyi segmentliyor* ve *hangisi gerçek zamanlı çalışabiliyor*. Üç mimari seçildi — biri semantic (SegFormer-B0), ikisi instance segmentation (RTMDet-Ins-tiny, YOLOv11n-Seg) — ve karşılaştırmanın mimariyi ölçmesi için veri bölünmesi, girdi hazırlığı, augmentasyon zinciri ve değerlendirme protokolü üçünde de sabit tutuldu.

Veri kaynağı BUSI (Breast Ultrasound Images): 780 görüntü, benign 437 / malignant 210 / normal 133. Çözünürlük 190×310 ile 1048×719 arasında değişiyor (780 görüntüde 639 farklı boyut), maskeler ikili ve `normal` sınıfının maskesi tamamen boş.

1.1 Kopya görüntüler

Veri setinde **115 teyitli kopya çifti** bulundu; 179 görüntü (%23) çok görüntülü kopya gruplarının içinde. Bunların 11'i "aynı görüntü, çelişkili maske" — ikisinde aynı kare biri "lezyon var" diğeri "lezyon yok" diye etiketlenmiş. Rastgele bölünmede bu görüntüler eğitim ve test'e dağılsa model ezberlediği kareyi tekrar görür ve Dice yapay olarak şişer.

Çözüm iki adımlı: çelişkili çiftlerde deterministik bir kuralla tek temsilci bırakıldı (780 → 769 görüntü) ve bölme **kopya grubu düzeyinde** yapıldı. Sonuç: eğitim 538, validation 116, test 115; **val ve test tamamen kopyasız**, her değerlendirme vakası ayrı bir sahne. Sınıf ve lezyon boyutu dağılımı üç bölümde de dengeli.

1.2 Değerlendirme protokolü

Üç modelin ortak metriği **orijinal çözünürlükte Dice**. Küçültülmüş uzayda karşılaştırmak semantic modele yapay avantaj verirdi, çünkü onun hedefi zaten küçültülmüş maske. Her modelin tahmini orijinal görüntü boyutuna taşınıp orijinal ikili maskeyle karşılaştırılıyor.

`normal` sınıfı bilerek veri setinde: canlı akışta karelerin çoğunda lezyon yok ve boş maskeler modele yanlış pozitif baskılamayı öğretiyor. Bu görüntülerde GT ve tahmin ikisi de boşsa Dice 1 sayılıyor; raporda lezyonlu ve boş kırılmı ayrı veriliyor.

Karar eşiği **validation'da** seçilip test'e sabit uygulandı. Test üzerinde eşik taraması yapılmadı — değerlendirme betiği test için eşiğin dışarıdan verilmesini zorunlu kılıyor ki bu yanlışlıkla ihlal edilemesin.

2. Kurulum

Girdi hazırlığı üçünde de **letterbox**: en-boy korunarak 256×256'ya küçültme, sağ-alta doldurma. Kare resize ilk denemede kullanılmıştı ve RTMDet'in maske geri-ölçekleme kodu tek ölçek katsayısı varsaydığı için maskeleri görüntüden kaydırıyordu; ölçüldüğünde maskenin yalnızca %59'u kendi tahmin kutusunun içine düşüyordu. Letterbox'a geçince sorun kaynağında kalktı.

Augmentasyon zinciri (yatay çevirme, affine, elastik deformasyon, parlaklık/kontrast, gamma, çarpımsal gürültü, hareket bulanıklığı) üç modelde ortak. Dikey çevirme bilerek yok: ultrasonda derinlik eksenı sabittir, dikey çevirme fiziksel olarak imkânsız görüntü üretir. CLAHE de yok — vakaların %75'i hipoeoik ve kontrast standardizasyonu bu sinyali bozar. RTMDet ve YOLO'nun kendi Mosaic/MixUp reçeteleri kapatıldı.

Anotasyon tarafında RTMDet ve YOLO aynı COCO JSON'undan besleniyor; YOLO etiketleri o JSON'dan türetildi ve geri-rasterize edildiğinde Dice **1.0** çıkıyor, yani dönüşüm kayıpsız. Böylece iki instance modeli birebir aynı anotasyonu görüyor.

3. Segmentasyon kalitesi

Aşağıdaki sayılar **test bölümünde**, her model kendi validation eşiğiyle. Dört konfigürasyon da yerelde, tek ortamda yeniden çalıştırıldı ve eğitim ortamındaki sonuçlar birebir yeniden üretildi (sapma 0.0000) — yani çıkarım yolu ile eğitim ortamı ayırmıyor.

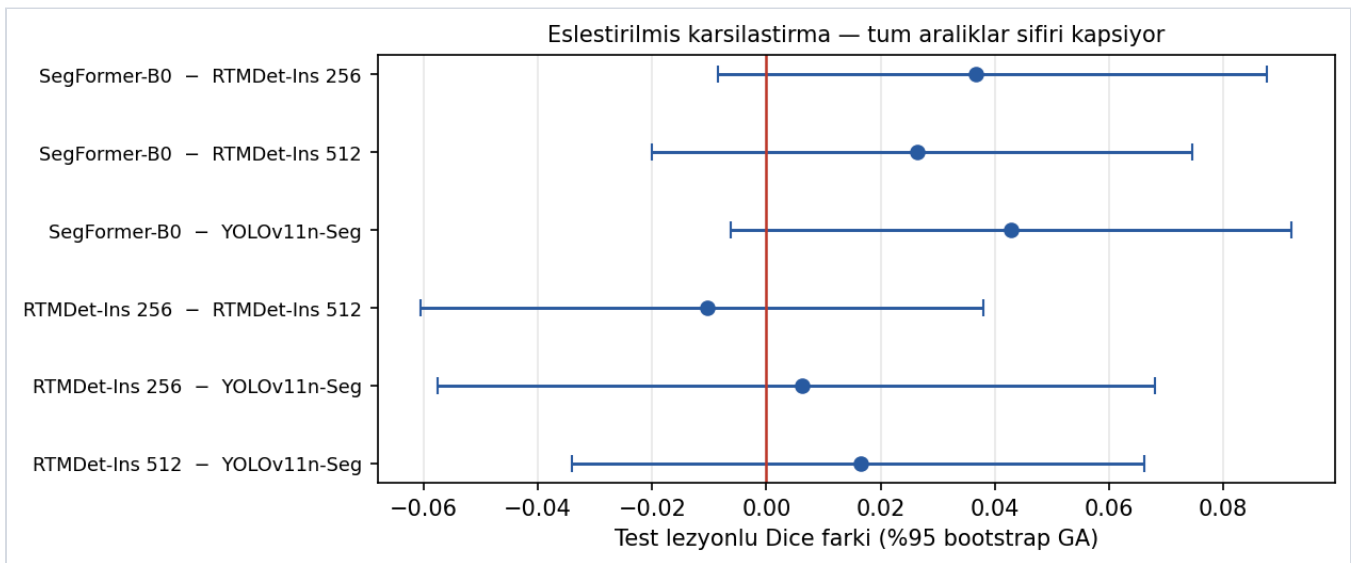
Model	Eşik	Test Dice (lezyonlu)	Test Dice (tüm)	IoU	Precision	Recall	Boş görüntü
SegFormer-B0	0.4000	0.7365	0.7800	0.6515	0.7603	0.7686	19/19
RTMDet-Ins 512	0.2000	0.7101	0.7406	0.6331	0.7198	0.7615	17/19
RTMDet-Ins 256	0.4000	0.6998	0.7407	0.6261	0.7241	0.7186	18/19
YOLOv11n-Seg	0.2000	0.6936	0.7268	0.6105	0.6664	0.7982	17/19

96 lezyonlu + 19 boş maskeli test görüntüsü. Eşikler validation'da `dice_tum` ölçütüyle seçildi.

3.1 Fark istatistiksel olarak anlamlı değil

Ortalamalar SegFormer-B0'ı öne koyuyor, ama aynı görüntüler üzerinde eşleştirilmiş karşılaştırma bunu desteklemiyor: **altı çiftin hiçbirinde güven aralığı sıfırı dışlamıyor.**

A	B	Fark	%95 GA	Wilcoxon p	A iyi / B iyi	Ayrışıyor
SegFormer-B0	RTMDet-Ins 256	0.037	-0.008 ... +0.088	0.618	38 / 50	hayir
SegFormer-B0	RTMDet-Ins 512	0.026	-0.020 ... +0.075	0.995	41 / 48	hayir
SegFormer-B0	YOLOv11n-Seg	0.043	-0.006 ... +0.092	0.010	54 / 35	hayir
RTMDet-Ins 256	RTMDet-Ins 512	-0.010	-0.060 ... +0.038	0.641	43 / 45	hayir
RTMDet-Ins 256	YOLOv11n-Seg	0.006	-0.058 ... +0.068	0.106	55 / 34	hayir
RTMDet-Ins 512	YOLOv11n-Seg	0.017	-0.034 ... +0.066	0.099	53 / 35	hayir



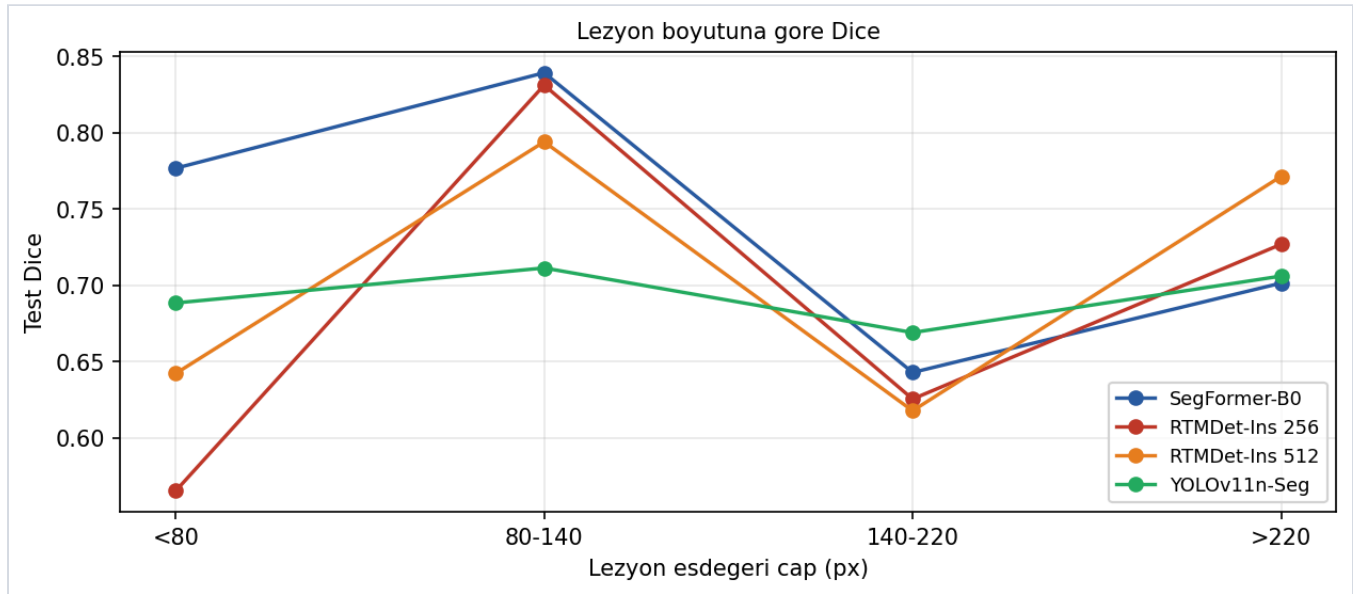
Test bölümünde ikili farklar ve %95 bootstrap güven aralıkları. Tüm aralıklar sıfır çizgisini kesiyor.

Kalite tarafında dört konfigürasyon ayırt edilemiyor. Tek koşuluk sıralama SegFormer-B0'ı öne koyuyor ancak fark ölçülen gürültü bandının (± 0.03 Dice) içinde. Sıralamayı kesinleştirmek tohum tekrarı gerektirir.

3.2 Modeller başarısızlık biçiminde ayrışıyor

Ortalama ile medyan ters yönü gösteriyor ve bu tek başına bilgi taşıyor: RTMDet tipik vakada biraz daha iyi (medyan her iki bölümde de önde), SegFormer kuyrukta daha sağlam. Test bölümünde Dice 0.3'ün altında kalan görüntü sayısı SegFormer'da 14, RTMDet 512'de 16, YOLO'da 17, RTMDet 256'da 18.

SegFormer'ın ortalama öndeliği buradan geliyor: çoğu görüntüde biraz geride, birkaç görüntüde çok önde. Klinik olarak bir lezyonu tamamen kaçırmak, sınırını birkaç piksel kaba çizmekten pahalı olduğu için bu ölçüt SegFormer'ı tercih ettirir — ama fark ortalamada anlamlı olmadığından bu bir eğilim, kanıt değil.



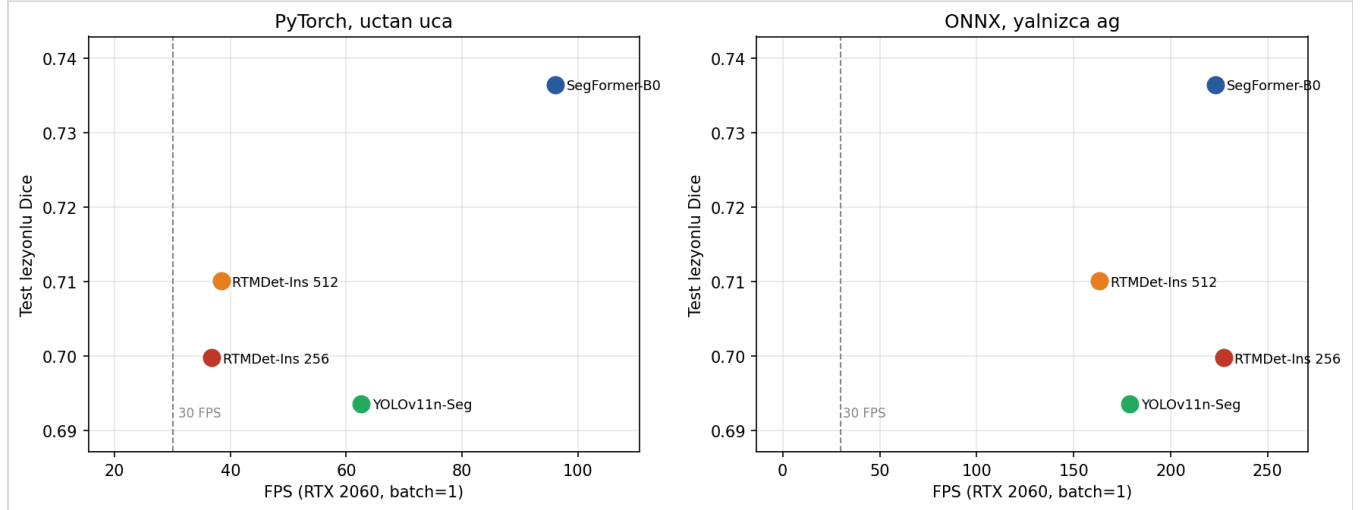
Lezyon büyüklüğüne göre test Dice. Dört model de 140–220 px bandında belirgin şekilde zayıflıyor; bu, veri setine özgü ortak bir zorluk.

4. Gerçek zamanlı ölçüm

Üç model üç farklı ortamda eğitildi, dolayısıyla hız ölçümü **tek makinede** tekrarlandı: RTX 2060, batch=1, ısınma sonrası, kare başına uçtan uca süre (ön işleme + ileri geçiş + maskeyi orijinal çözünürlüğe taşıma). Diskten görüntü okuma dışarıda — canlı akışta kare cihazdan gelir. Ölçüm betiği, hızı ölçülen kodun kaliteyi ölçen kodla aynı olduğunu doğruluyor.

Model	Param (M)	PyTorch ms	PyTorch FPS	Saf ağ ms	Çerçeve yükü	ONNX GPU ms	ONNX CPU ms	VRAM (MB)
SegFormer-B0	3.71	10.41	96.10	9.20	%12	4.48	22.03	63.00
YOLOv11n-Seg	2.84	15.96	62.60	10.73	%33	5.59	7.42	33.80
RTMDet-Ins 512	5.62	26.03	38.40	13.62	%48	6.12	44.21	77.70
RTMDet-Ins 256	5.62	27.15	36.80	13.32	%51	4.40	11.87	88.90

Tekrarlanan koşullarda GPU süreleri %5–10 oynuyor; modeller arası farklar bunun üzerinde ama küçük farklar anlamlı okunmamalı.



Kalite-hız ödünleşimi. Solda PyTorch uçtan uca, sağda ONNX (yalnızca ağ). Kesikli çizgi 30 FPS. Dört konfigürasyon da her iki ölçümde gerçek zamanlı bandın üzerinde.

4.1 Girdi boyutu gecikmeyi neredeyse hiç etkilemiyor

RTMDet 256 ile 512 aynı süreyi veriyor (27.2 vs 26.0 ms), saf ağ ölçümünde de öyle — dört kat piksele rağmen. Sebep, bu boyutlarda batch=1'de GPU'nun kernel başlatma yükünde takılı olması, hesap yükünde değil. CPU ölçümü bunu doğruluyor: orada girdi boyutu fark yaratıyor (94.6 vs 48.2 ms), çünkü CPU hesap yüküne bağlı.

Pratik sonucu: 256 ile 512 arasında ne Dice farkı var ne FPS farkı. 512'nin tek somut maliyeti eğitimdeki bellek (2102 MB vs 606 MB).

4.2 Çerçeve yükü kaldırılınca sıralama değişiyor

RTMDet'in gecikmesinin yarısı mmdet'in kare başına Python yükü (saf ağ 13.3 ms, uçtan uca 27.2 ms). SegFormer'da bu oran %12, YOLO'da %33. Üç ağ da ONNX'e aktarılıp aynı çalışma zamanında ölçüldüğünde

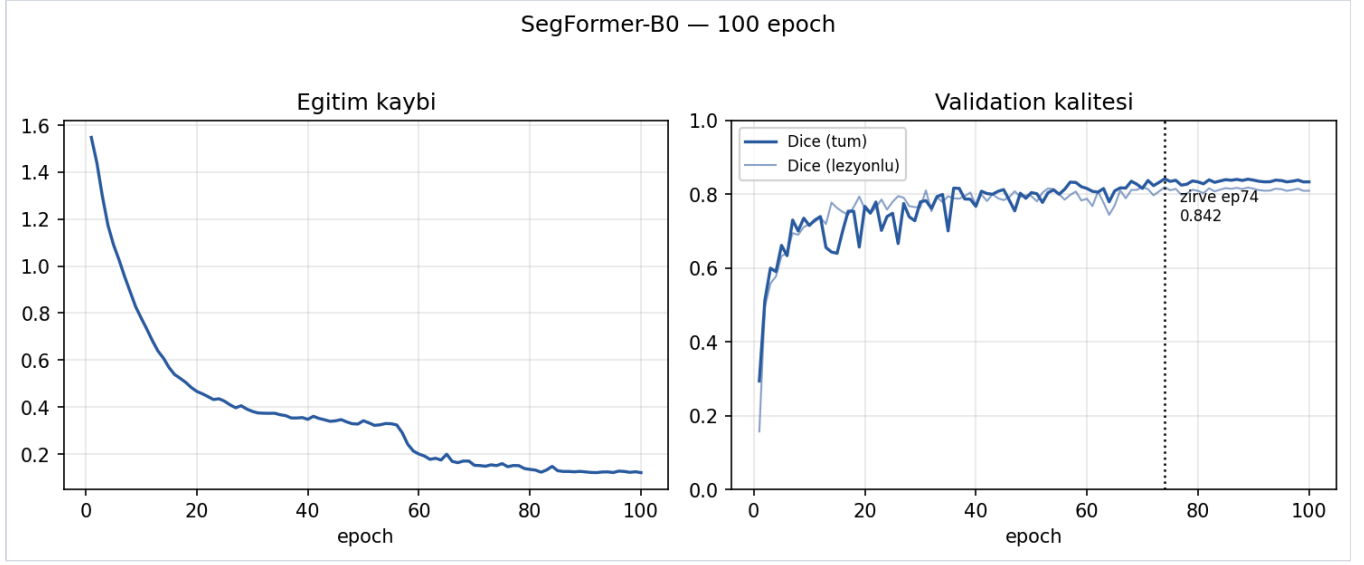
RTMDet-Ins-tiny 256 en hızlısı çıkıyor (4.40 ms). Export'ların sayısal doğruluğu bağıl hata ile kontrol edildi: 5e-7 ... 1.3e-5, fp32 yuvarlama düzeyinde.

Yani PyTorch ölçümüne bakarak mimari sıralaması yapmak yanıltıcı olurdu; "RTMDet 2.6 kat yavaş" cümlesi mimariyi değil çerçeveyi ölçüyor.

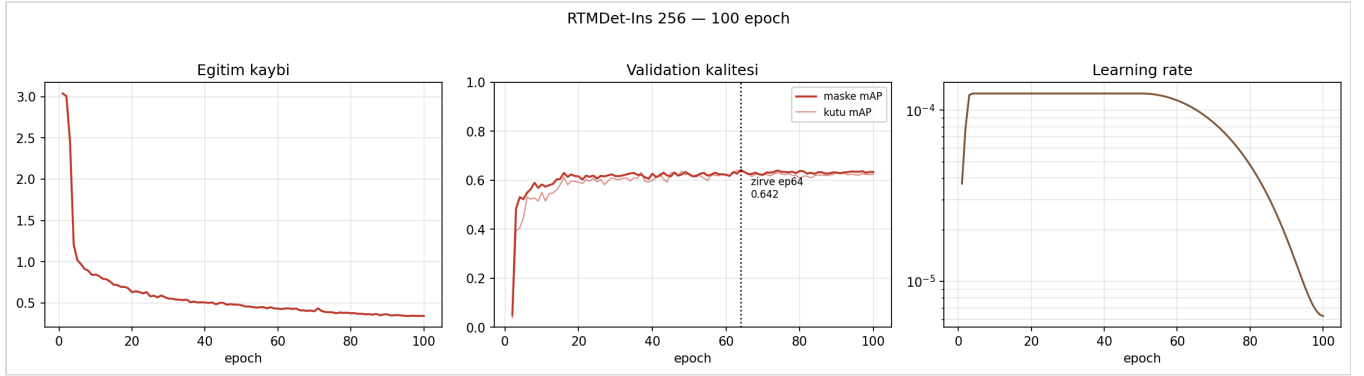
CPU'da YOLOv11n-Seg açık ara önde: ONNX ile 7.4 ms, yani GPU'suz bir cihazda bile 135 FPS. Gerçek zamanlı hedef için ayırık GPU'nun zorunlu olmadığını gösteriyor.

ONNX sayıları **dağıtılabilir FPS değil:** grafik yalnızca ağı içeriyor. Instance modellerde son işleme (NMS + maske birleştirme) dışarıda ve ayrıca ölçüldü (RTMDet'te 2.8 ms / 1.6 ms), SegFormer'ınki eşikleme + yeniden boyutlandırmadan ibaret. Uçtan uca dağıtımda SegFormer'ın avantajı bu tablodan büyük, RTMDet'inki küçük.

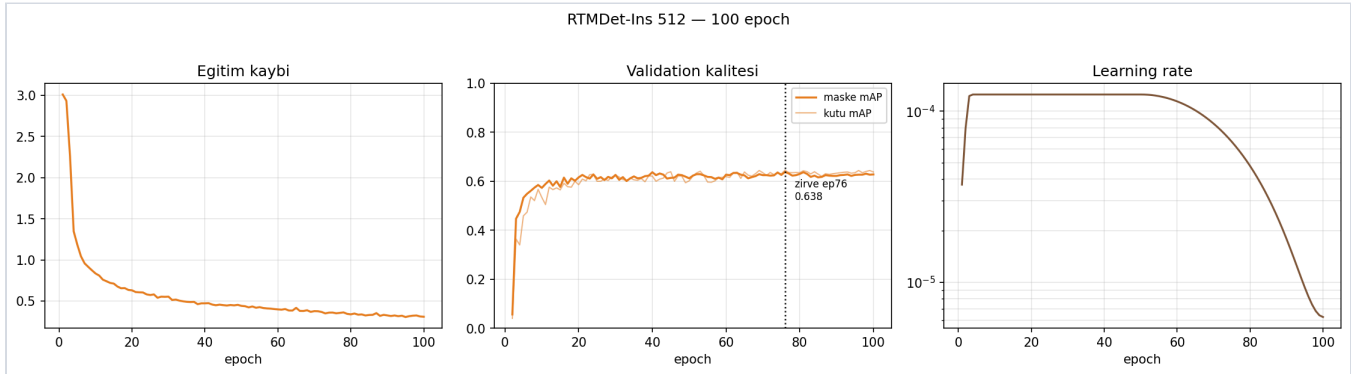
5. Model bazında eğitim davranışı



SegFormer-B0 — 100 epoch. Validation Dice epoch 74'te zirve (0.842), sonrasında düz. Seçim ölçütü raporlanan metriğin kendisi: eşik taranarak hesaplanan dice_tüm.



RTMDet-Ins 256 — 100 epoch. Zirve epoch 64 (segm mAP 0.642). Maske mAP kutu mAP'inin üzerinde; bu veri setinde beklenen sıralama.



RTMDet-Ins 512 — 100 epoch. Zirve epoch 76 (0.638). 256 ile pratikte aynı yere yakınıyor.

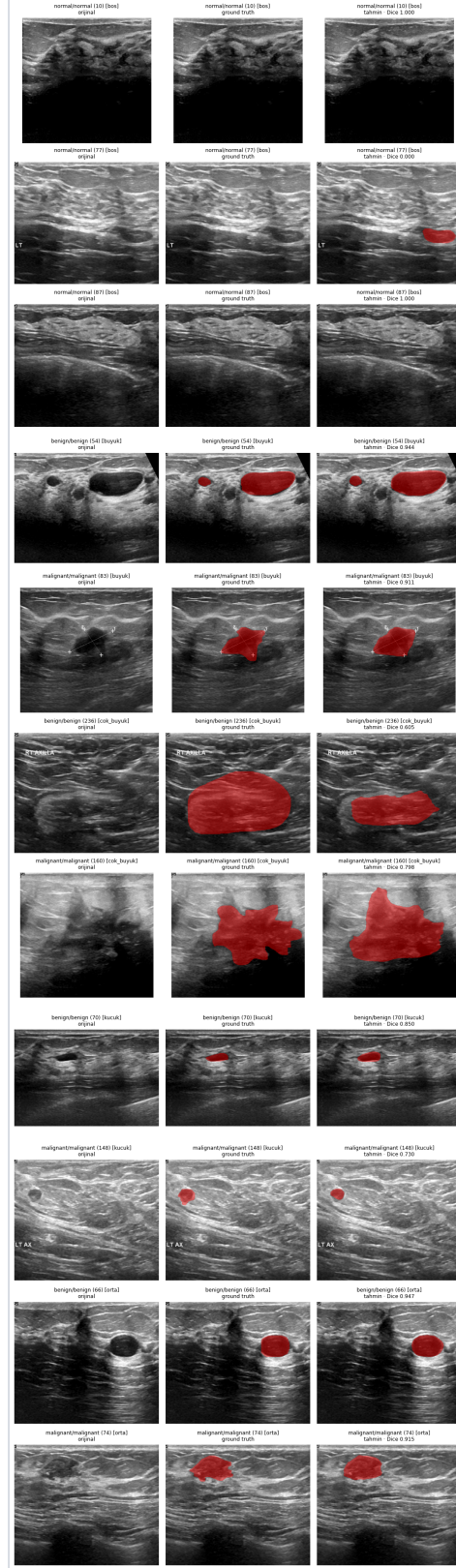
Eğrilerde validation kaybı bilerek yok. SegFormer ve RTMDet'te validation adımı kayıp değil kalite metriği üretiyor (Dice / mAP) — mmdet validation'ı tahmin modunda çalıştırıyor, SegFormer döngüsü de aynı yapıyı izliyor. Ultralytics validation kaybını hesaplıyor ama tek modelde gösterip diğer ikisinde göstermemek eğrileri karşılaştırılmaz hale getirdi.

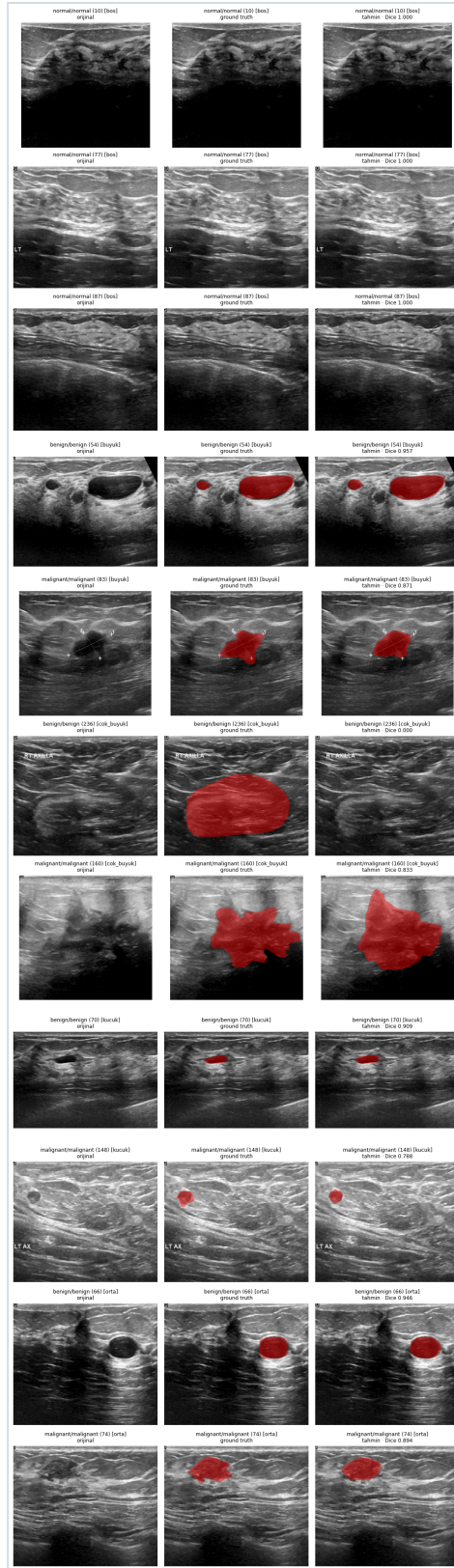
Kayıp bilgisi zaten modeller arası okunamaz: üç mimarının kayıp fonksiyonu farklı (Dice+BCE / cls+bbox+mask / box+seg+cls+df). Val kaybının asıl işi aşırı öğrenmeyi göstermek ve bunu validation kalite eğrisi daha doğrudan yapıyor — üç eğride de orta panelde.

YOLOv11n-Seg'in eğitim eğrisi bu sürümde yok: Ultralytics'in `results.csv` log dosyası rapor üretilirken elde değildi. Modelin validation mAP seyri epoch 60 civarında plato yapıyor (eğitim çıktısından). Dosya eklendiğinde figür `figurleri_uret.py` ile otomatik üretiliyor.

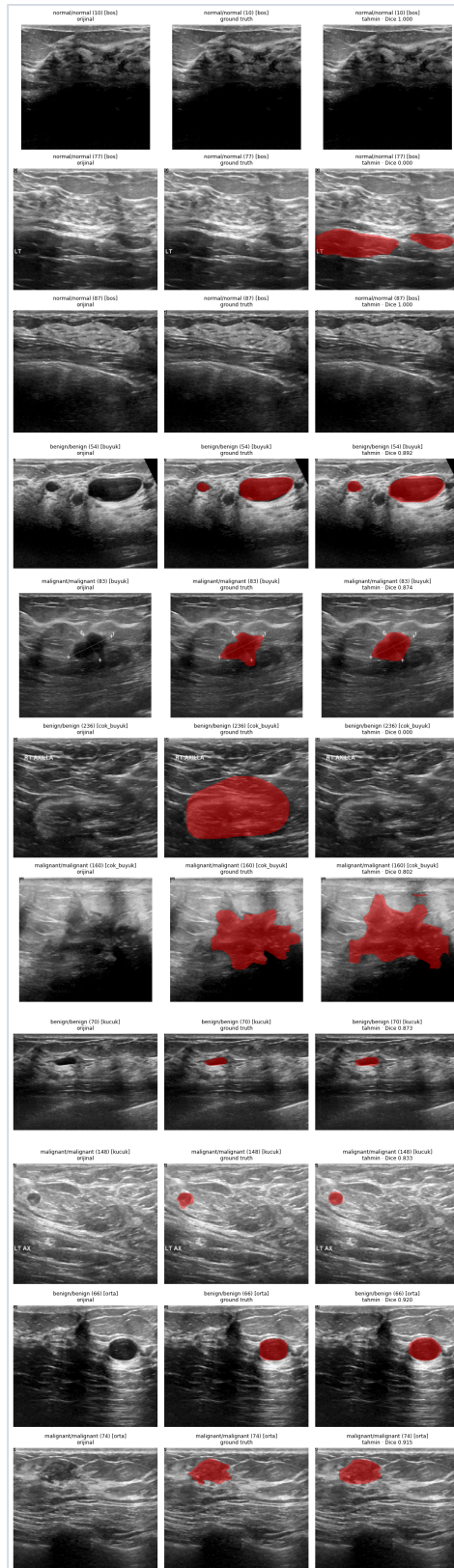
6. Sabit değeriendirme vakaları

Aynı 11 vaka üç modelde de gösteriliyor: sınıf × lezyon boyutu kovalarından seçilmiş 8 lezyonlu ve 3 boş maskeli görüntü. Her model kendi iyi örneğini seçeydi görsel karşılaştırma hiçbir şey söylemezdi.





RTMDet-Ins 256 — aynı vakalar. Üç normal görüntünün hepsinde temiz; buna karşılık çok büyük lezyonlu benign (236) vakasını tamamen kaçırıyor (SegFormer aynı vakada 0.605 alıyor).



YOLOv11n-Seg — aynı vakalar.

7. Ölçüm sırasında düzeltilen üç hata

Bu bölüm rapora bilerek konuluyor: üçü de sessizce yanlış sonuç üretiyordu ve ikisi bir süre yanlış yorumlanmıştı.

7.1 segm_mAP'i girdi boyutuna bağlı olarak düşüren maske hatası

RTMDet-Ins maskeyi orijinal boyuttan 1 piksel kısa üretebiliyor (mmdet'te ters çevrilmiş ölçek katsayılarının indisleri takas edilmiş). COCO değerlendirmesi iki maskenin boyutu uyuşmayınca o görüntüyü sıfır sayıyor. Etkilenen görüntü oranı girdi boyutuna bağlı: 256'da val görüntülerinin %21.6'sı, 512'de %1.7'si. Sonuç, 256'nın `segm_mAP` 'i 0.412 görünürken gerçekte 0.626 olması.

Bu, "512 mimari olarak çok daha iyi" gibi okunuyordu ve o okuma yanlış olurdu. Düzeltildikten sonra iki koşu arasında anlamlı fark kalmadı. İkinci bedeli checkpoint seçimiydi: bozuk metrikle seçilen epoch 40 yerine düzeltilmiş metrikle epoch 64 seçildi ve Dice 0.803'ten 0.808'e çıktı.

7.2 Ölçümle reddedilen bir "düzeltme"

`cv2.INTER_NEAREST` maskeyi küçültürken yarım piksel kaydırıyor. Bunu düzeltmenin ~0.01 Dice kazandıracağı düşünülüp mmdet yamalandı. **Ölçüm reddetti:** aynı config, 100 epoch, tek değişken yama — yamalı koşu her metrikte daha kötü çıktı. Sebep, mmdet'in çıkarım tarafının ters yönde bir kayma taşımaması ve nearest'in bias'ının onu telafi ediyor olmasıydı; tek ucu düzeltmek tutarlılığı bozdu. Yama geri alındı.

Buradan kalıcı bir kontrol çıktı: tahmini ± 2 piksel kaydırıp Dice'in nerede zirve yaptığına bakan, modelden bağımsız bir hizalama testi. Dört konfigürasyonda da kaydırma kazancı ihmal edilebilir (0.0000–0.0012); reddedilen yamalı koşuda 0.0043 ile açıkça işaretleniyor.

7.3 Boş görüntüleri saymayan eşik seçimi

Eşik ilk olarak yalnızca lezyonlu görüntülerdeki Dice'a bakılarak seçiliyordu. YOLO'da bu, 20 normal görüntünün 8'inde yanlış pozitif üreten bir eşiği seçti. Ölçüt boş görüntüleri de sayan `dice_tum` 'e çevrildi; YOLO'nun eşiği 0.05'ten 0.20'ye taşındı ve yanlış pozitif 1'e düştü. RTMDet'te iki ölçüt de aynı eşiği veriyor, yani değişiklik yalnızca farkın önemli olduğu yerde etkili oldu.

8. Sonuç

"En iyi model" sorusunun cevabı dağıtım hedefine bağlı.

- **Kalite:** dört konfigürasyon istatistiksel olarak ayrışmıyor. SegFormer-B0 ortalamada önde ve ağır başarısızlıkları en az; RTMDet tipik vakada biraz daha iyi.
- **PyTorch, olduğu gibi dağıtım:** SegFormer-B0 (96 FPS) — mmdet'in çerçeve yükü RTMDet'i geride bırakıyor.
- **ONNX + GPU:** RTMDet-Ins 256 ile SegFormer-B0 ayrışmıyor (4.4 ms).
- **GPU'suz cihaz:** YOLOv11n-Seg açık ara (ONNX CPU, 135 FPS).

Tek bir öneri gerekiyorsa **SegFormer-B0**: her ölçümde ilk ikide, en az ağır başarısızlık, boş görüntülerde en temiz ve dağıtımı en basit (instance son işlemesi yok).

8.1 Bu raporun sınırları

- **Her model tek kez eğitildi.** Kalite sıralaması bu yüzden kapatılamıyor; kapatmanın yolu tohum tekrarı.
- **Checkpoint seçim ölçütü üç modelde farklı** (SegFormer'da Dice, RTMDet'te mAP, YOLO'da Ultralytics fitness). SegFormer lehine küçük bir avantaj olabilir; ara epoch ağırlıkları saklanmadığı için geriye dönük ölçülemiyor.
- **Augmentasyon paritesi tam değil:** elastik deformasyon YOLO'da yok, çünkü Ultralytics'in albumentations sarmalayıcısı maskeleri taşımıyor.
- **Test bölümü küçük:** 96 lezyonlu + 19 boş görüntü. Boş görüntülerdeki "19/19 vs 17/19" farkı iki görüntü demek.
- **ONNX ölçümü ağ ile sınırlı;** uçtan uca dağıtım hattı ayrıca ölçülmeli.